efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

Piles et Files

TI301 – P2 2025/2026

Nicolas FLASQUE

Prérequis :

Savoir implémenter les LSC, les listes HT et les tableaux
Réaliser des chaînages en tête et en queue
Justifier la complexité des opérations de chaînage

Objectifs :

Définir **pile** et **file** comme des types ‘abstraits’
Définir les opérations élémentaires pour pile et file
Implémenter pile et file à partir de tableaux, LSC et listes HT(réutilisation)
Expliquer la meilleure implémentation en termes de performance

Organisation :

1 CM de 2h
1 séance TD de 2h
1 séance de TP

Piles : présentation

Pile

Je dépose au-dessus

IN

Je retire au-dessus

OUT

La dernière que je dépose (Last In) sera la première que je retire (First Out) : LIFO

Files : présentation

J'arrive en fin de file

IN

Je repars en début de file

OUT

Le premier qui est arrivé (First In) sera le premier à repartir (First Out) :

FIFO

Piles et files : types abstraits

En POO, on décrit des **comportements** et des **états**

Pas d'implémentation pour le moment

Objectifs : ensemble des opérations / fonctions à utiliser

Implémentation ensuite

Exemple pour une pile : comportements

Une pile peut être vide ou pas ; **‘pile vide ?’**

On peut afficher tous les éléments d’une pile : **‘afficher’**

On peut ajouter un élément : **‘empiler’ – push()**

On peut retirer un élément : **‘dépiler’ – pop()**

On peut consulter le prochain élément accessible : **‘consulter’ – top()**

On peut créer une pile : **‘créer’**

Étape suivante : comportements ->éléments d'algorithmique : exemple pour une pile

opération	A besoin de	Modifie l'état ?	Valeur attendue ?
pileVide	Pile	non	Vrai/faux
afficher	Pile	non	Rien
Empiler - push	Pile, valeur	oui	Rien
Dépiler – pop	Pile	oui	Valeur dépilée
Consulter - top	Pile	non	Valeur consultée
créer	Rien	oui	Pile

Indépendant du type des éléments dans la pile (entier, réel, caractère…)

Un peu de vocabulaire

opération	Nom classique
pileVide	isEmptyStack
afficher	displayStack
empiler	push
dépiler	pop
consulter	top
créer	createStack

Un prototype en algo, puis en C

opération	A besoin de	Modifie l'état ?	Valeur attendue ?
isEmptyStack	pile	non	Vrai/faux

int **isEmptyStack**(Pile p);

Pour aller plus loin : choisissons un type

Pile pour stocker des **éléments de type T**

opération	A besoin de	Modifie l'état ?	Valeur attendue ?
pop	Pile	oui	Valeur dépilée

T **pop**(pile *p);

Pour implémenter en C : choisissons un type

Pile pour stocker des **caractères**

opération	A besoin de	Modifie l'état ?	Valeur attendue ?
pop	Pile	oui	Valeur dépilée

```
char pop(pile *);
```

Piles : rappel

Pile

Je dépose au-dessus

IN

Je retire au-dessus

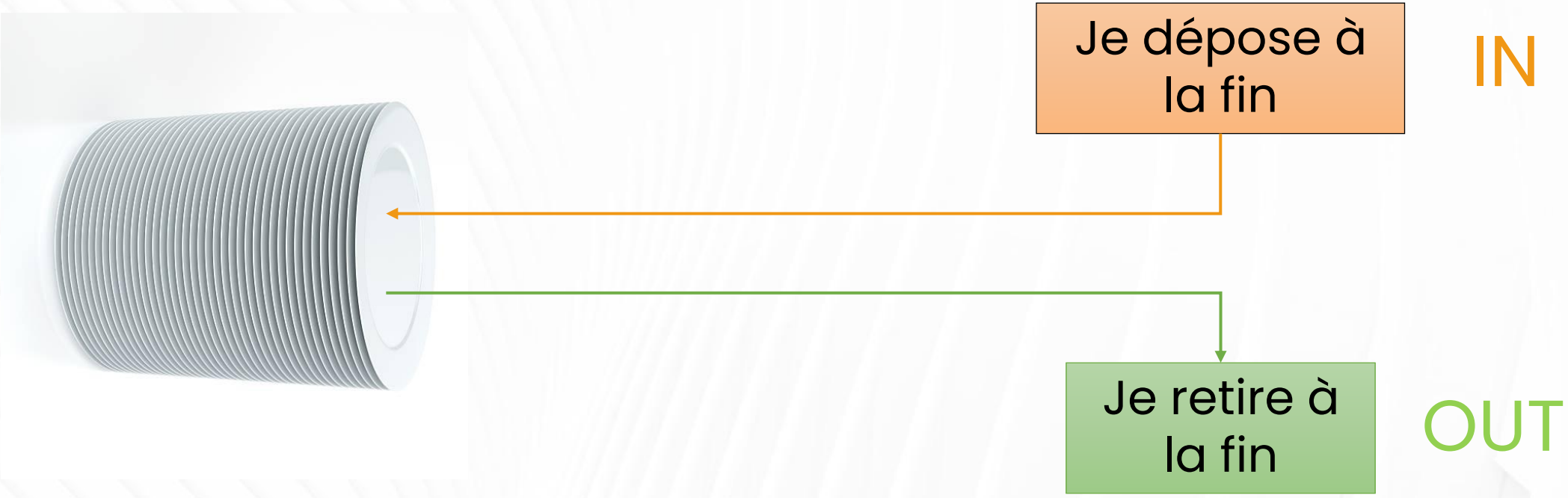
OUT

Piles : travail à partir du schéma

Pile

Piles : travail à partir du schéma

Pile



Pile : principe d’accès

On peut donc empiler/ajouter ou dépiler/retirer au 'même endroit', à la 'même extrémité' (début ou fin) d'une structure qui stocke des valeurs

Quelles structures de données sont adaptées à ces traitements ?

Accès systématique au début : **liste**

Accès systématique à la fin : **tableaux (liste ht ?)**

Pile et listes :

Les éléments sont chaînés en tête de liste

Le dernier inséré (Last In) est le premier accessible pour 'sortir' (First Out)

Push : Empiler une valeur= ajouter une nouvelle cellule en tête de liste

Pop : Dépiler une valeur = retirer la cellule en tête de liste
(+ garder sa valeur)

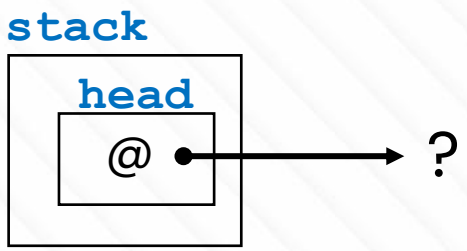
Piles et listes

On définit le type pile comme une liste `t_list` :

```
typedef t_list t_stacklist
```

On stocke donc juste la valeur du pointeur `'head'`

Soit la variable `t_stacklist stack;` sa représentation est :



En anglais , **Pile** se traduit par **Stack**

Piles et listes

Tester si une pile est vide = tester si la liste est vide - déjà fait
Une pile n'est jamais pleine avec les listes
Empiler un élément = ajouter en tête de liste – déjà fait
Dépiler un élément = retirer en tête de liste – presque déjà fait
Consulter un élément = regarder la valeur en tête de liste – presque déjà fait

Vu en TD/TP

Pile et tableaux :

Les éléments sont insérés à 'la fin' du tableau

Le dernier inséré (Last In) est accessible pour 'sortir' (First Out)

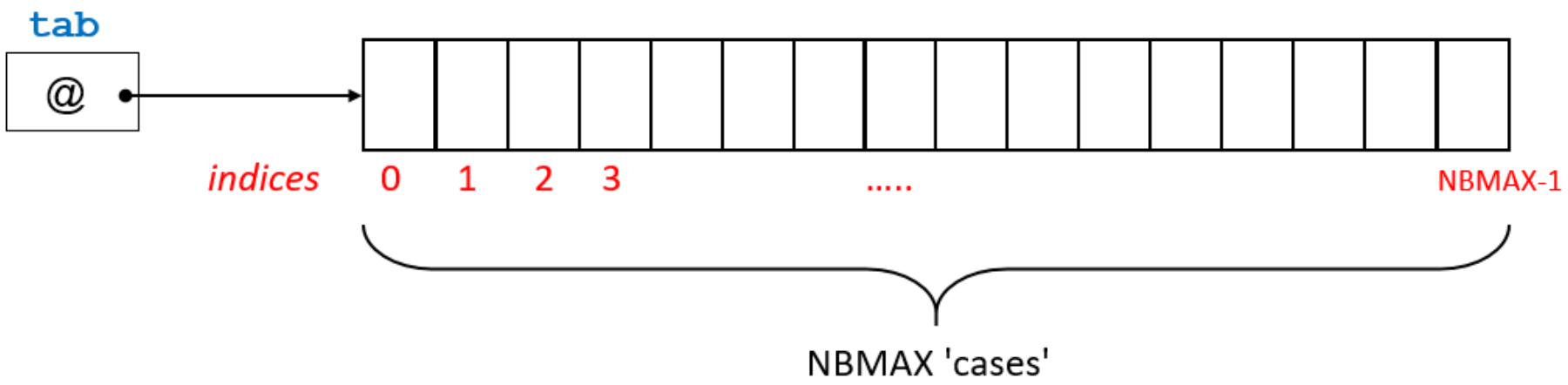
Push : Empiler = ajouter un élément à la fin du tableau

Pop : Dépiler = retirer l'élément qui est à la fin du tableau

Pile et tableaux : illustration

Soit un tableau **`tab`**, et sa taille maximum **`NBMAX`**

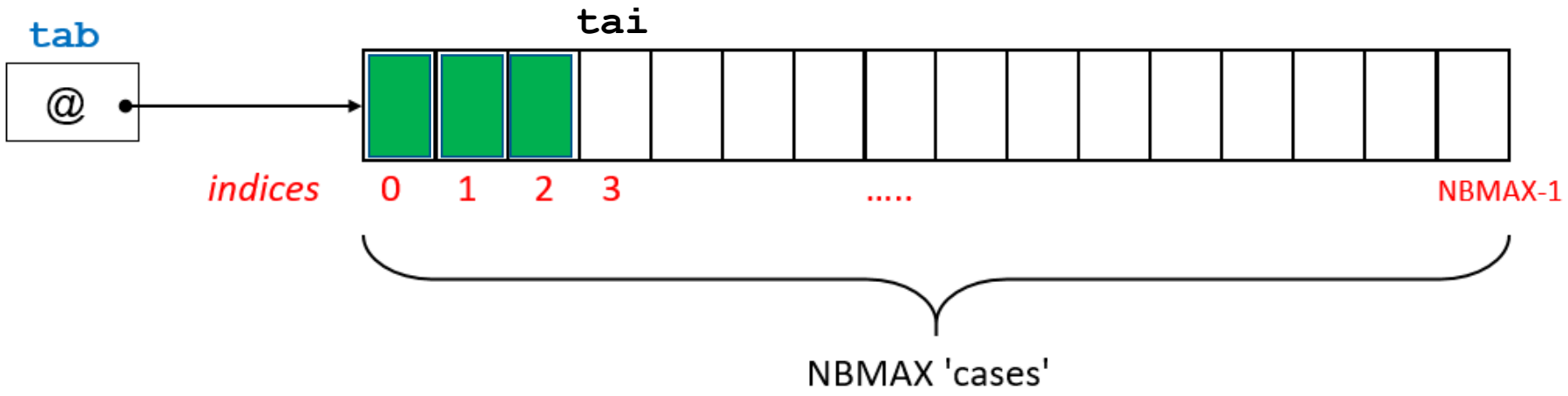
`NBMAX` indique combien on peut stocker au maximum d'éléments dans le tableau (taille fixe)



Pile et tableaux : illustration

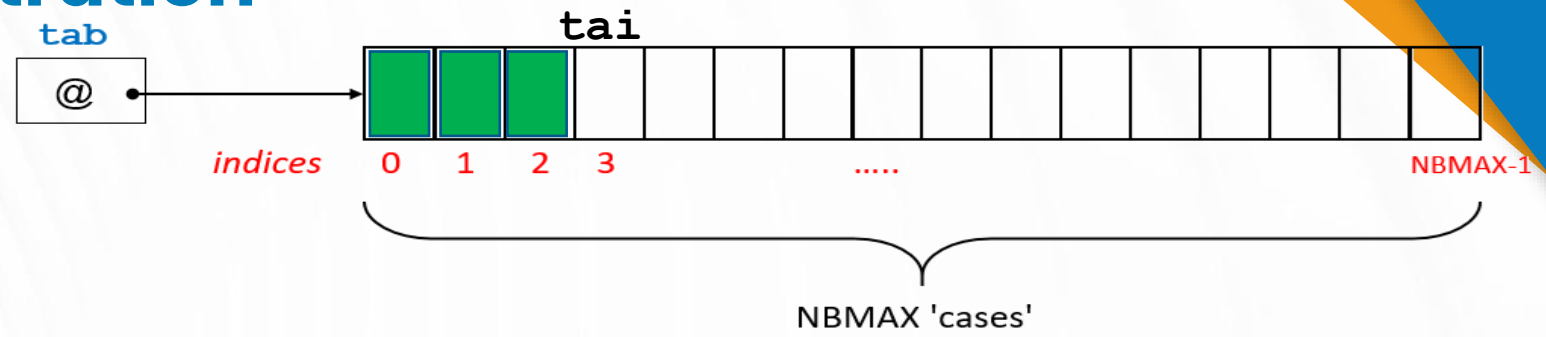
On utilise une taille 'logique' **`tai`** pour savoir combien de cases sont 'utilisées'.
Notons en vert les cases 'utilisées', c'est-à-dire qui contiennent des valeurs connues . Sur l'exemple suivant, on en utilisera 3. **`tai=3`**

On représente cette variable comme un indice sur le schéma



Pile et tableaux : illustration

tai = 3

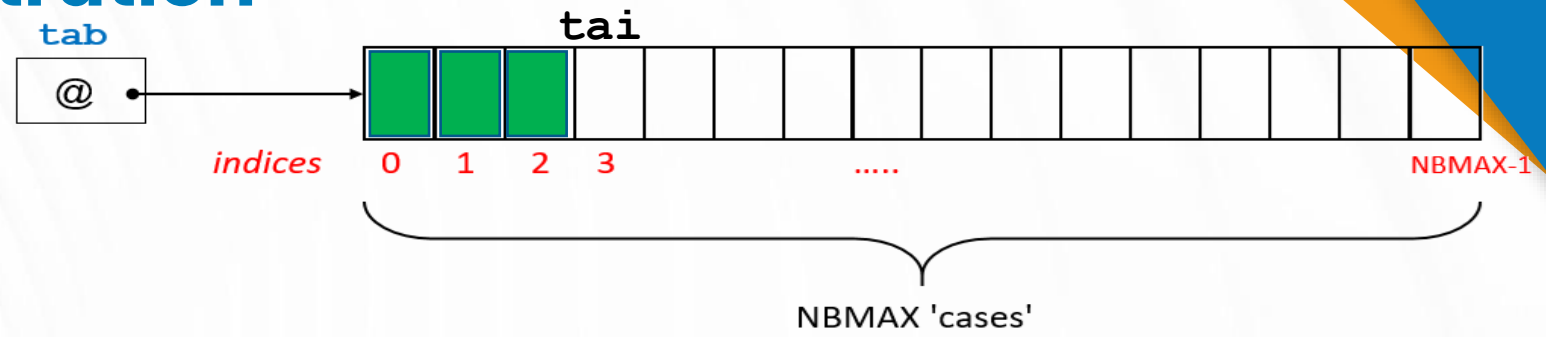


Push = ajouter une valeur à 'la fin' = à la prochaine 'case' disponible

Quel est l'indice de cette case ?
C'est 3, en fait c'est **tai**
Quelle est la complexité ?
O(1), on connaît l'indice -> accès direct et immédiat à la donnée

Pile et tableaux : illustration

`tai` = 3



Pop = retirer une valeur à 'la fin' = à la dernière case utilisée

Quel est l'indice de cette case ?
C'est 2, en fait c'est `tai-1`
Quelle est la complexité ?
O(1), on connaît l'indice -> accès direct à la donnée

Pile : résumé

Si on utilise ...	alors Push() est	Et Pop() est
Une liste	Ajouter en tête de liste	Retirer en tête de liste *
Un tableau	Ajouter dans la prochaine case disponible *	Retirer de la dernière case utilisée *

* : offre soumise à conditions

Quelle condition pour 'Retirer en tête de liste' et 'Retirer de la dernière case utilisée' ?
Il y a quelque chose à retirer = **la pile n'est pas vide**

Quelle condition pour 'Ajouter dans la prochaine case disponible' ?
Il reste de la place dans le tableau = **la pile n'est pas pleine**

Rappel : avec une liste, on considère qu'il n'y a pas de taille maximale (la seule limite est la mémoire disponible)

Pile et tableau : éléments d'implémentation

Pour définir une pile à l'aide d'un tableau, on définit une valeur NBMAX

```
#define NBMAX 50
```

Et une structure, qui stockera le tableau, et sa taille logique :

```
struct s_stacktab
{
        int values[NBMAX];        // en algo on utiliserait T pour un type quelconque
        int nbElts;               // nbElts correspond à tai
};

typedef struct s_stacktab t_stacktab;
```

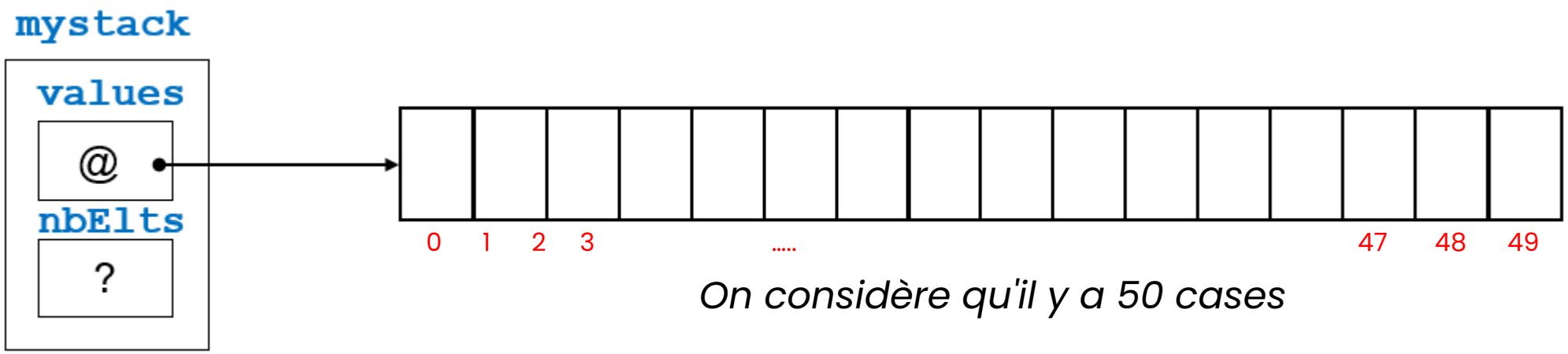
Pile et tableau : représentation du type

Soit la définition de variable suivante :

```
t_stacktab mystack;
```

Rappel : `values` et `nbElts` sont les **champs** de la variable `mystack`

La représentation en est :



On considère qu'il y a 50 cases

isEmptyStack() et isFullStack()

A quelle condition peut-on dire que la pile est vide ?

Si le tableau ne stocke aucun élément : le champ `nbElts` est égal à 0

Pour en faire une fonction :

nom	A besoin de	Modifie l'état ?	Valeur attendue ?
isEmptyStack()	Pile à tester	non	Vrai/faux (1/0)

nom | **paramètre(s)** | **le paramètre est-il un pointeur ou non ?** | **type de retour**

fonction isEmptyStack()

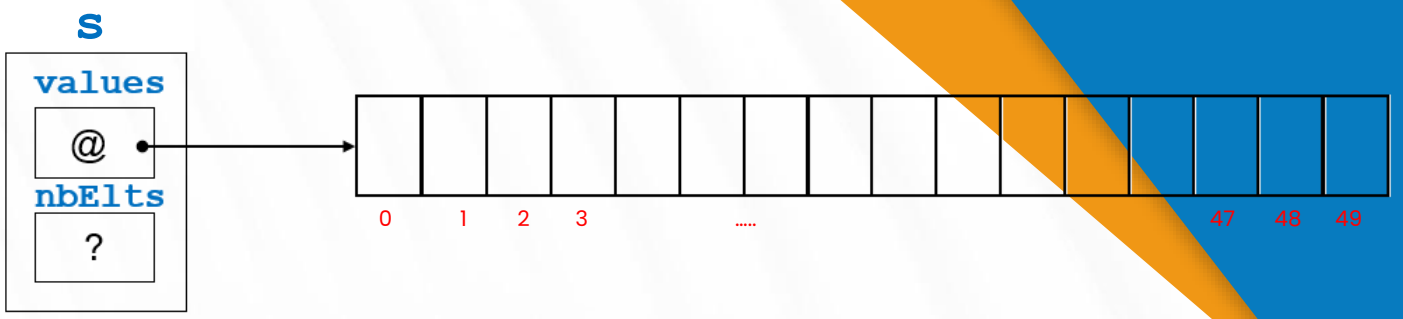
nom	A besoin de	Modifie l'état ?	Valeur attendue ?
isEmptyStack()	Pile à tester	non	Vrai/faux (1/0)

nom **paramètre(s)** **le paramètre est-il un pointeur ou non ?** **type de retour**

D'où l'entête :

```
int isEmptyStack(t_stacktab);
```

Fonction : isEmptyStack()



Puis sa définition :

```
int isEmptyStack(t_stacktab s)
{
    int res, nbElems;
    nbElems = s.nbElts;

    if (nbElems == 0)
    {
        res = 1;
    }
    else
    {
        res = 0;
        }

    return res;
}
```

En version compacte

```
int isEmptyStack(t_stacktab s)
{
    return (s.nbElts==0);
}
```

Dépiler : pop()

On souhaite retirer une valeur `val` à une pile `mystack`, et la conserver

A quelle condition peut-on retirer une valeur de la pile ?

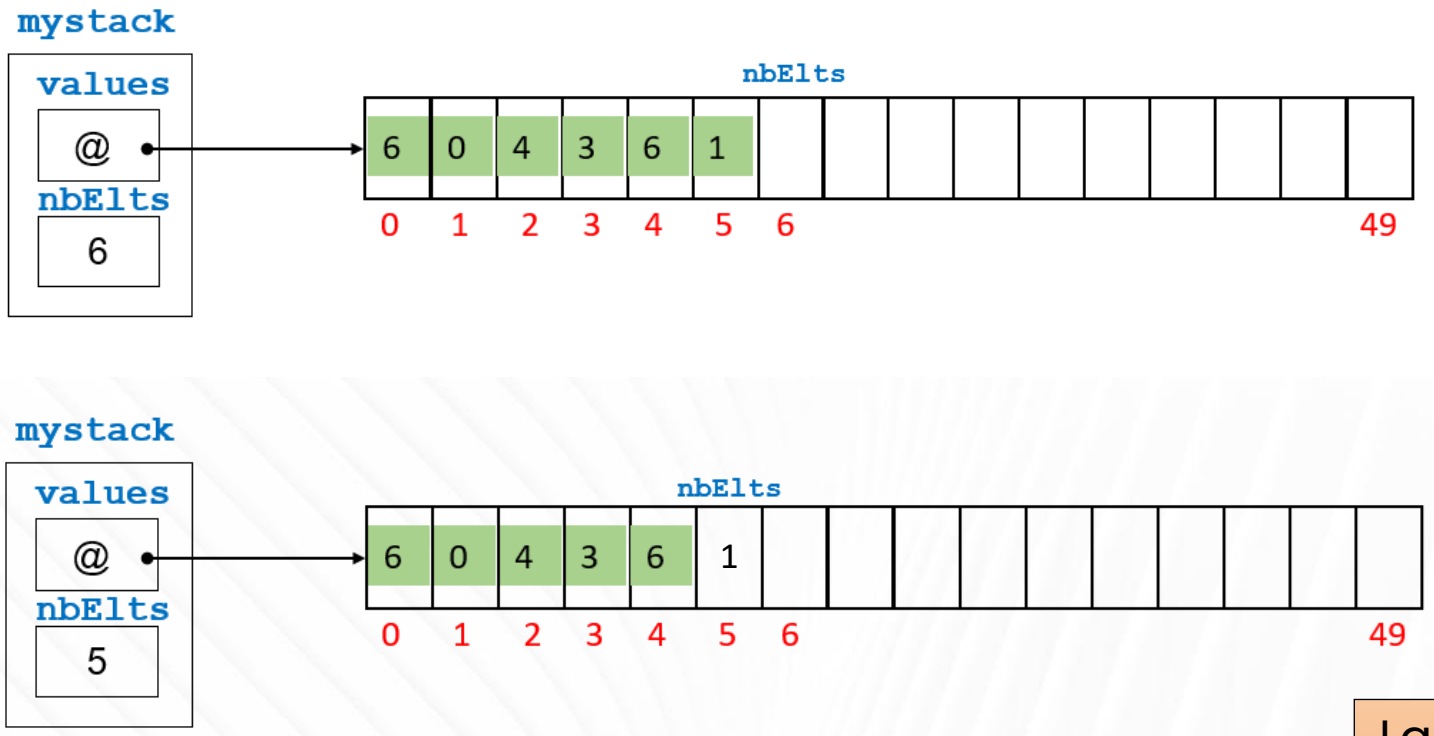
Si la pile n'est pas vide

Quel est l'effet du dépilement si la pile n'est pas vide ?

On récupère la valeur de la dernière case utilisée, et on note que cette case n'est plus utilisée

Dépiler : pop() - illustration

Soit la pile **mystack** dans l'état suivant :



Effets du dépilement :

- ✓ On récupère la valeur 1
- ✓ On indique qu'il y a un élément en moins

La valeur 1 est toujours dans le tableau, mais la pile a diminué de taille

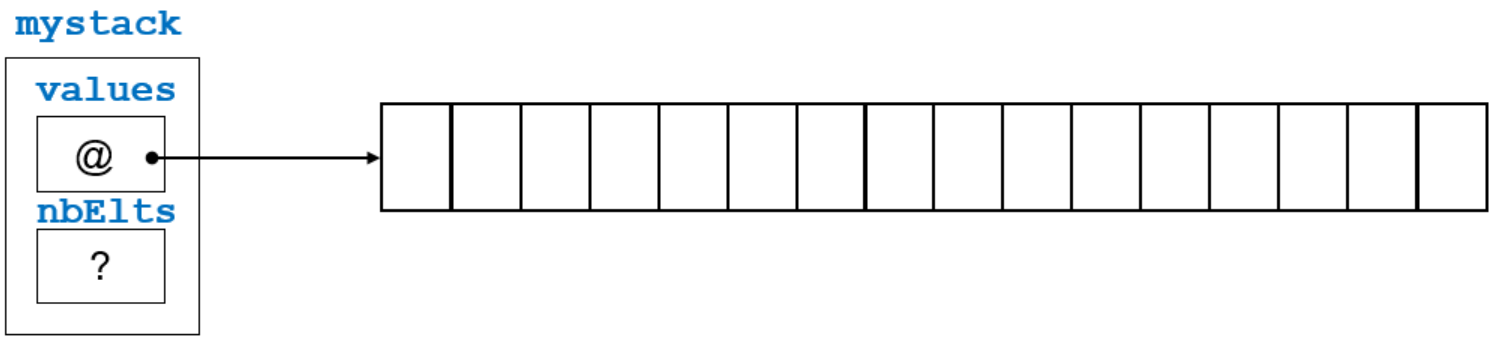
1

fonction : pop()

nom	A besoin de	Modifie l'état ?	Valeur attendue ?
pop()	Pile à traiter	oui	La valeur dépilée

nom **paramètre(s)** **le paramètre est-il un pointeur ou non ?** **type de retour**

Savoir si `pop()` modifie la pile = savoir si la variable (ses champs) est modifiée



`nbElts` est modifié : donc il y a bien modification

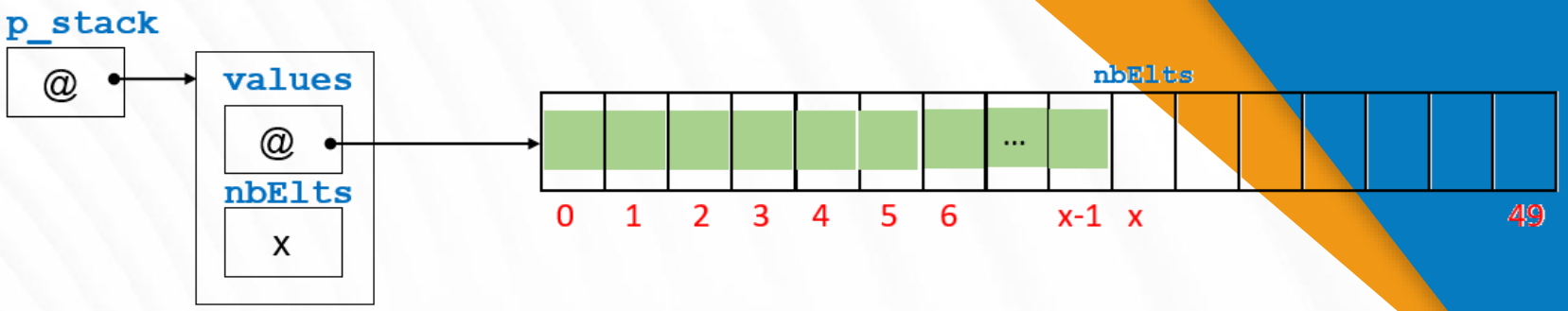
fonction pop()

fonction	A besoin de	Modifie l'état ?	Valeur attendue ?
pop()	Pile à traiter	oui	La valeur dépilée

nom **paramètre(s)** **le paramètre est-il un pointeur ou non ?** **type de retour**

D'où le début de la fonction

```
T pop(t_stacktab *);
```



Implémentation

```
int pop(t_stacktab *p_stack)
{
    int valeur;
    int position;

    position = p_stack->nbElts;

    if (position > 0)     // ou isEmptyStack()==0
        valeur = p_stack->values[position-1];
        p_stack->nbElts = position -1;
    else // si la pile est vide ??? Que retourner ???
            ?????                // vu en en TD/TP

return valeur;
}
```

A traiter en TD / TP

`isFullStack()`

La fonction `push()` pour empiler, avec un tableau

Files : rappel

Je dépose à la fin

IN

Je retire au début

OUT

Un peu de vocabulaire

En anglais, file se dit ‘queue’

Ajouter, enfiler un élément se dit ‘enqueue’

Retirer, défiler un élément se dit ‘dequeue’

Un peu de vocabulaire

opération	Nom classique
fileVide	isEmptyQueue
afficher	displayQueue
enfiler	enqueue
défiler	dequeue
consulter	top
créer	createQueue

File : principe d’accès

On ajoute à une extrémité, on retire à l'autre extrémité.

Quelles structures de données sont adaptées à ces traitements ?

Listes avec pointeurs 'head' et 'tail' (`t_ht_list`)

Tableaux ! Avec un indice pour le début (`first`) et la fin (`last`)

File et listes `head` + `tail`

Question : où faire les 'ajouts' (enqueue) et 'retraits' (dequeue) de manière efficace dans une telle liste ?

Le choix est :

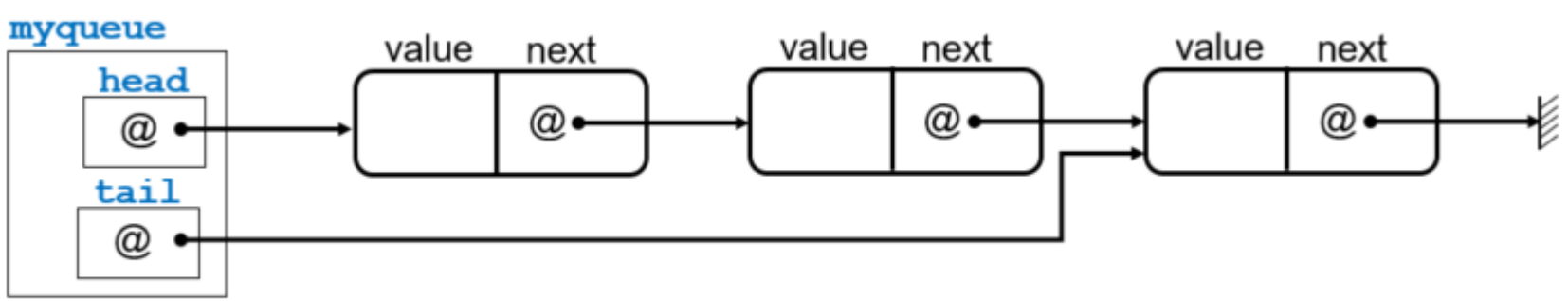
1) J'ajoute au début (`head`), je retire à la fin (`tail`)

Ou

2) J'ajoute à la fin (`tail`), je retire au début(`head`)

Option 1 :J'ajoute au début (`head`), je retire à la fin (`tail`)

Soit la file définie par `t_queue myqueue`



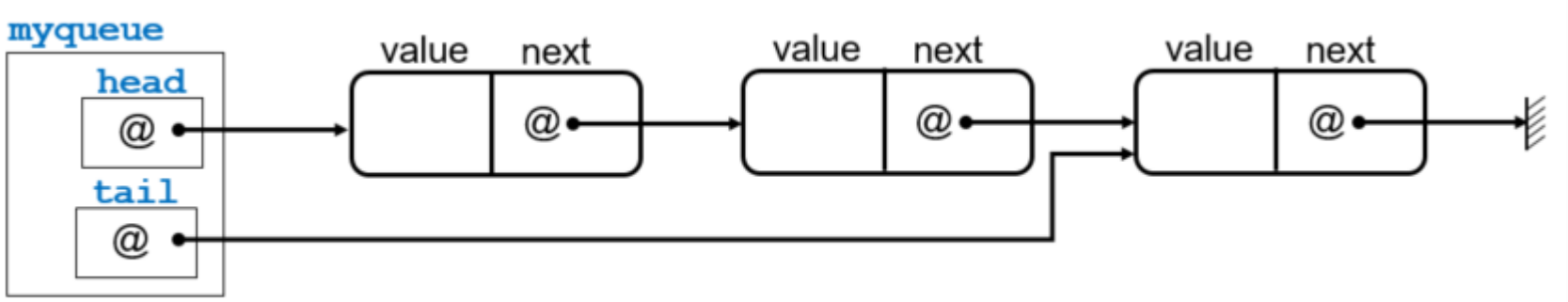
Ajouter au début d'une liste ht : déjà fait, simple

Retirer à la fin d'une liste ht : que faudrait-il faire ?
Parcourir toute la liste jusqu'à l'avant dernière pour retirer la dernière !

La complexité de 'retirer à la fin' est $O(N)$

Option 2 :J'ajoute à la fin (`tail`), je retire au début (`head`)

Soit la file définie par `t_queue myqueue`



Ajouter à la fin d'une list ht : déjà fait, simple

Retirer au début d'une liste ht : déjà vu pour les piles, simple

Files et listes

Tester si une file est vide = tester si la liste ht est vide - déjà fait
Une file n'est jamais pleine avec les listes
Enfiler un élément = ajouter en queue de liste ht – déjà fait
Défiler un élément = retirer en tête de liste ht–déjà fait
Consulter un élément = regarder la valeur en tête de liste ht – presque déjà fait

Vu en TD/TP

File et tableaux :

Les éléments sont insérés à la ‘fin’ du tableau et retirés au ‘début’

Le premier inséré (First In) est accessible pour 'sortir' (First Out)

Enqueue : Enfiler = ajouter un élément à la fin du tableau

Dequeue : Défiler = retirer l'élément qui est au début du tableau

File et tableaux : éléments d'implémentation

Pour définir une file à l'aide d'un tableau, on définit :

Une taille physique (maximale, 50 sur les exemples)

Un indice de 'fin' de tableau pour enfiler les éléments : `last`

Un indice de 'début' de tableau pour défiler les éléments : `first`

File et tableaux : éléments d'implémentation

D'où la définition de structure (en langage C) suivante :

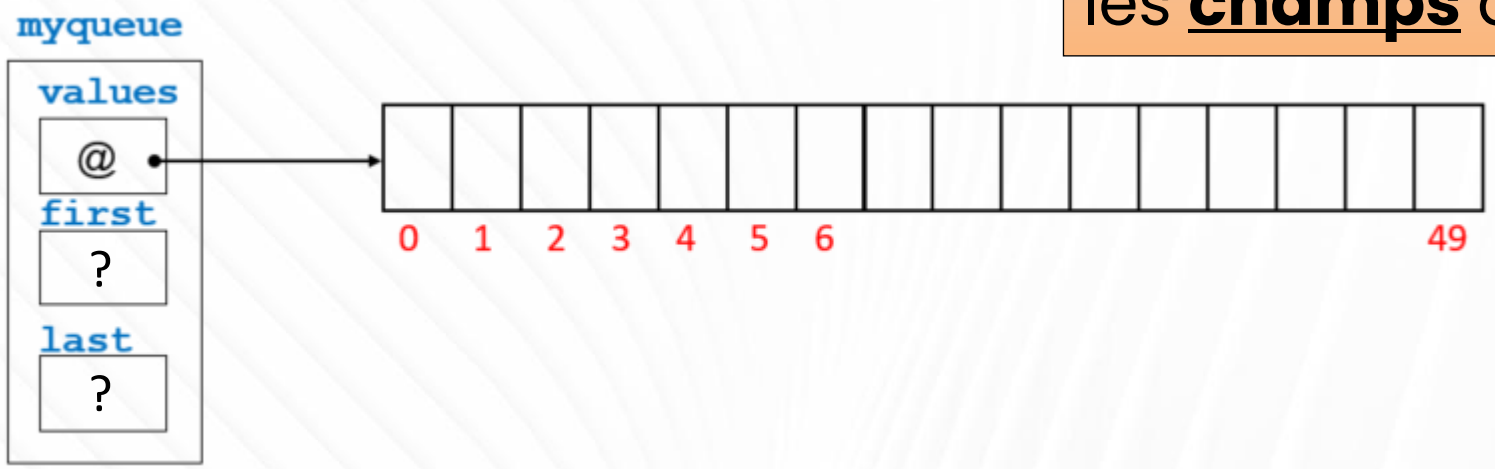
```
#define NBMAX 50

struct s_queuetab
{
        int values[NBMAX];      // remplacer int par T en algo
        int first, last;        // les indices pour les opérations
};

typedef struct s_queuetab t_queuetab;
```

File et tableau : représentation du type

Soit la définition de variable suivante : `t_queue myqueue;`

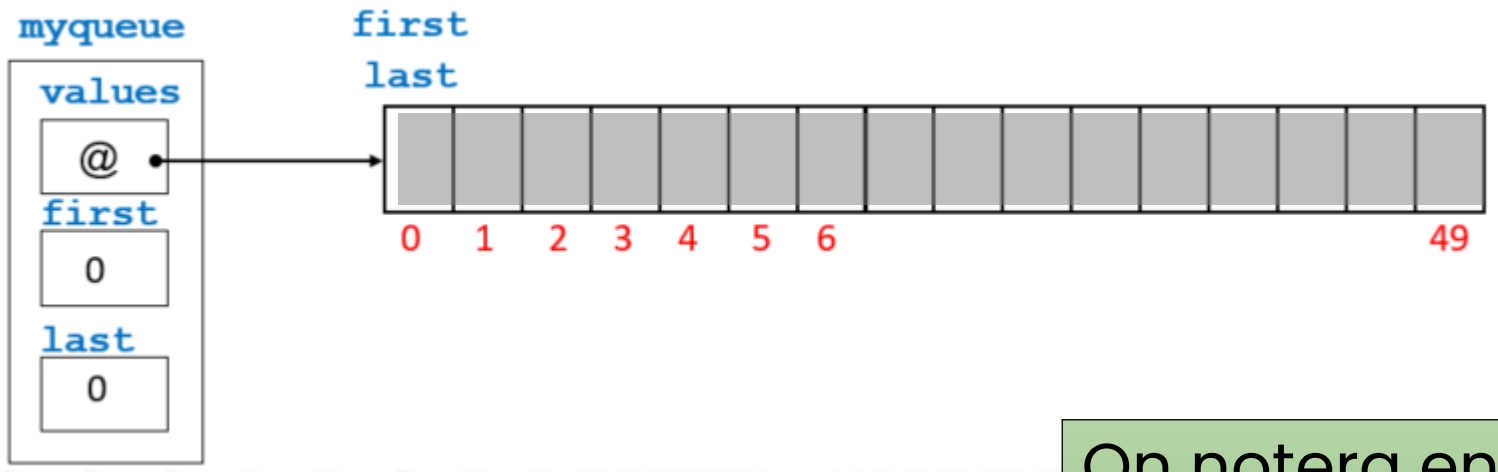
La représentation en est :

Rappel : `values`, `first` et `last` sont les **champs** de la variable `myqueue`



Créer une file vide

S'il n'y a aucun élément, first et last seront mis à 0
La représentation en est :



On notera en gris les cases 'non utilisées' du tableau

`first` : où défiler (dequeue)
`last` : où enfiler (enqueue)

Enfiler (enqueue) le premier élément

Pour ajouter un élément, `last` indique l'indice de la case où l'insérer

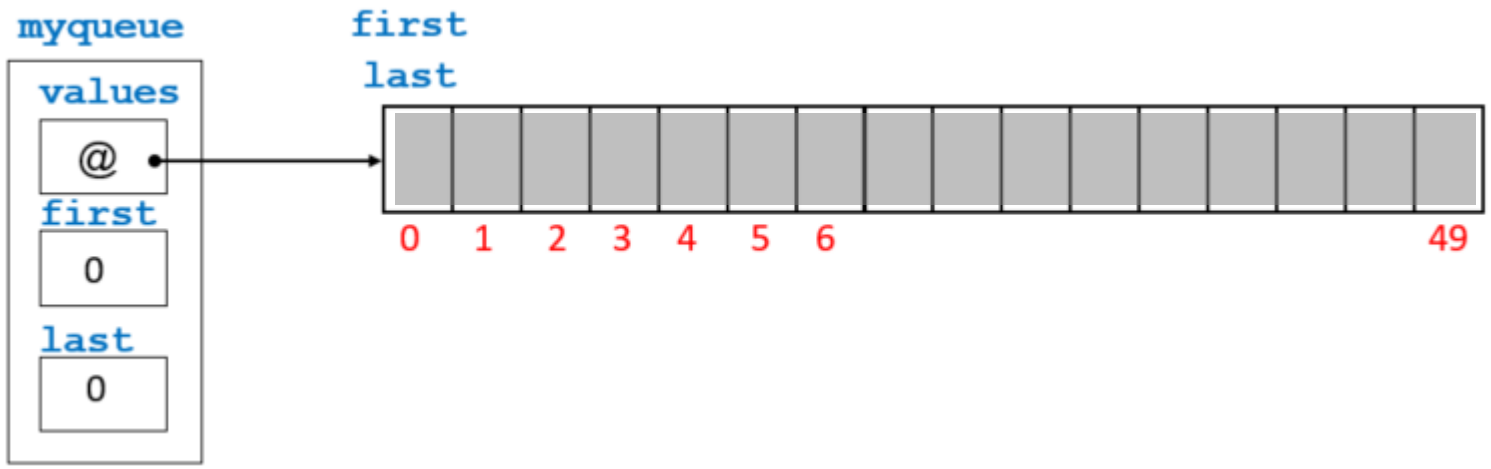
Il faut ensuite indiquer que le prochain élément à enfiler occupera la case suivante : `last` doit augmenter

Question : combien d'éléments peut-on 'enfiler' au maximum ?

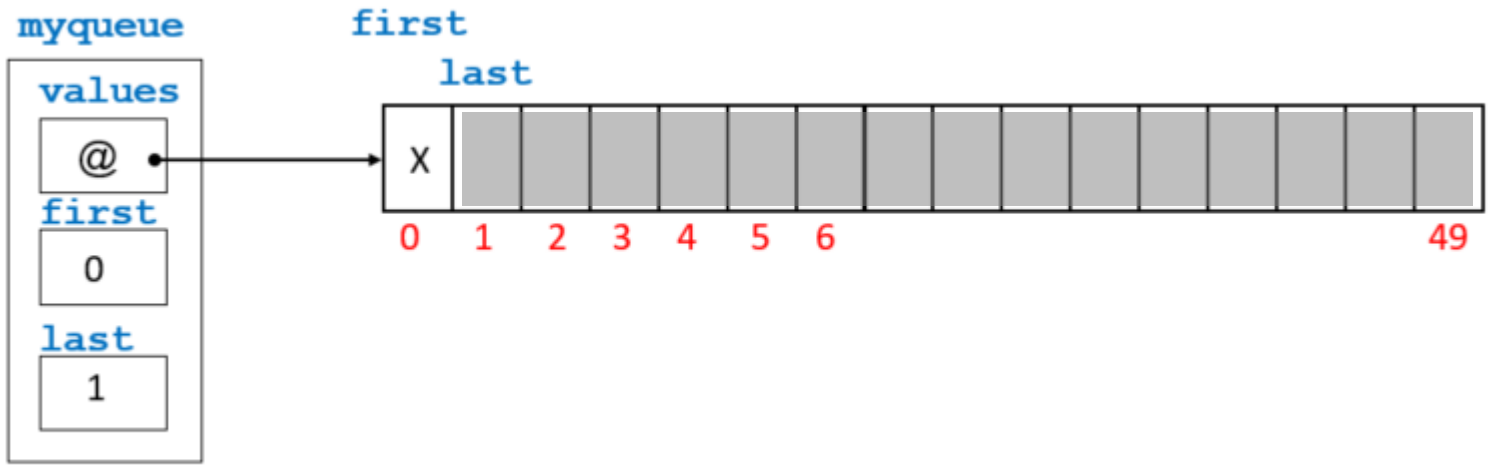
50, soit la taille du tableau

Illustration

Avant enqueue



Après enqueue
d'un élément 'X'



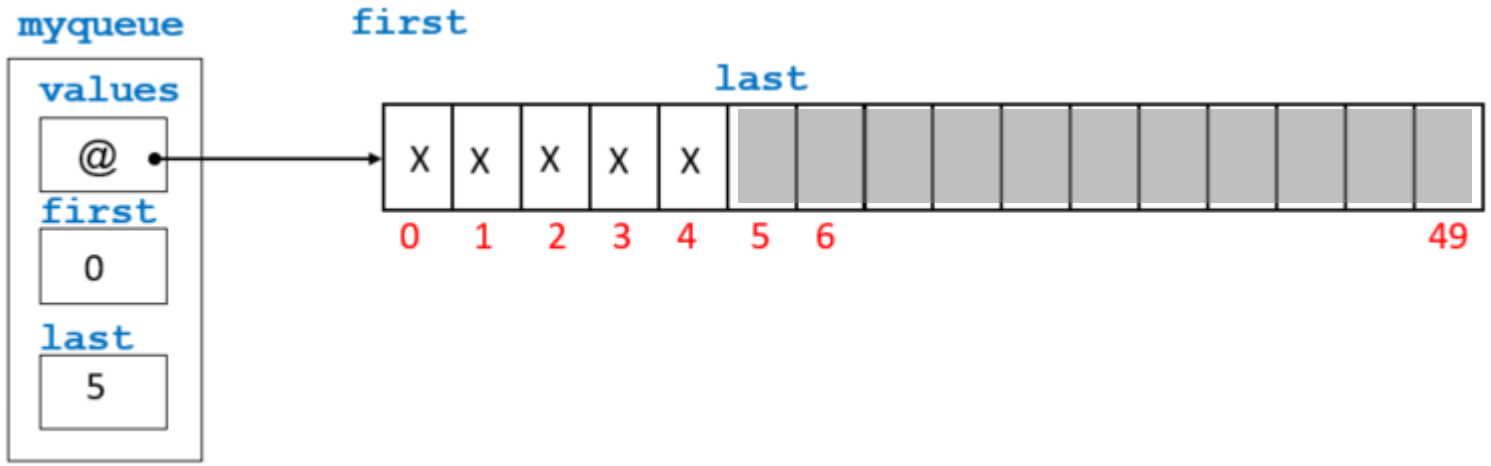
Dequeue (enlever l'élément mis en premier)

Les éléments sont rangés dans l'ordre d'arrivée.

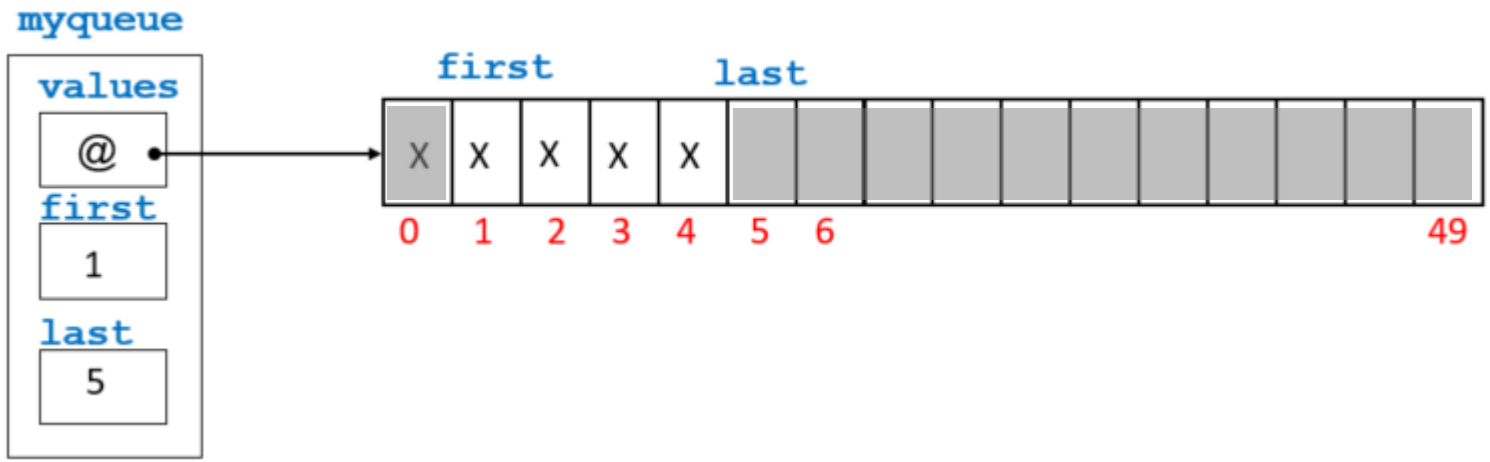
Quand on enfile un élément, `last` est modifié, mais pas `first`, qui désigne bien là où se trouve l'élément rangé en premier

Donc pour défiler : on récupère l'élément dans la case d'indice `first`, puis on indique que le prochain à défiler est dans la case suivante, on augmente `first`.

Avant dequeue



Après dequeue :
on récupère la valeur
en case 0



Constat

On utilise maintenant une partie du tableau qui ne débute plus à l'indice 0, mais à celui stocké dans `first`

C'est une gestion classique de tableau avec des boucles, sauf que l'on commence la boucle à `first` au lieu de 0…

isEmptyQueue()

Comment peut-on déterminer qu'une file (avec tableau) est vide ?

Quand toutes ses cases sont inutilisées

Exemple :

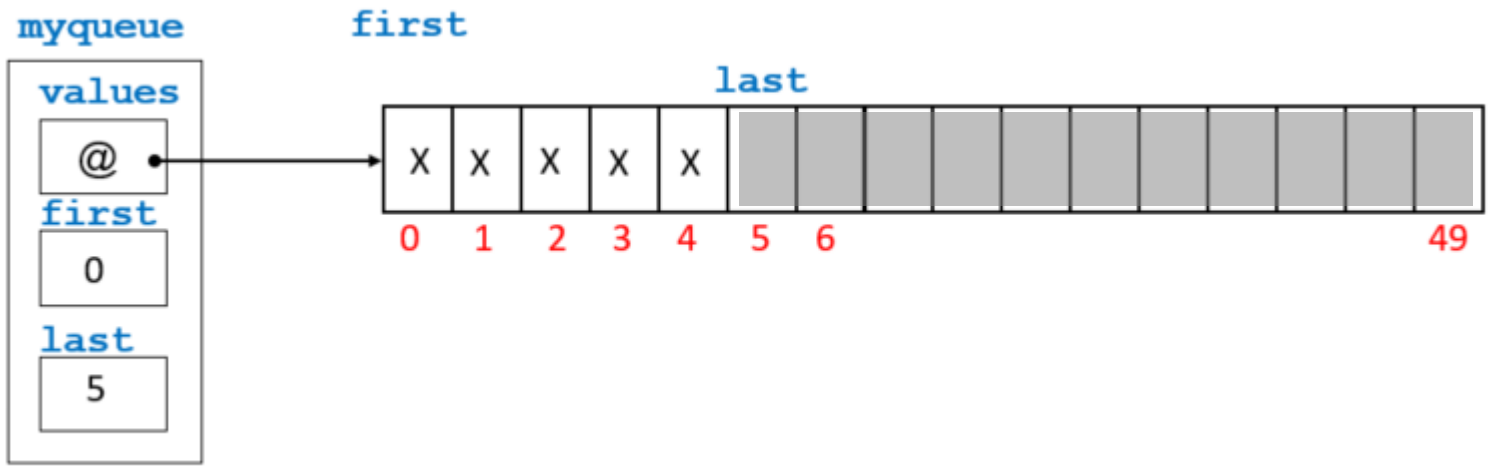
on enfile 5 éléments, puis on défile 5 éléments

Combien d'éléments dans la file ? Il n'en reste aucun : la file est vide

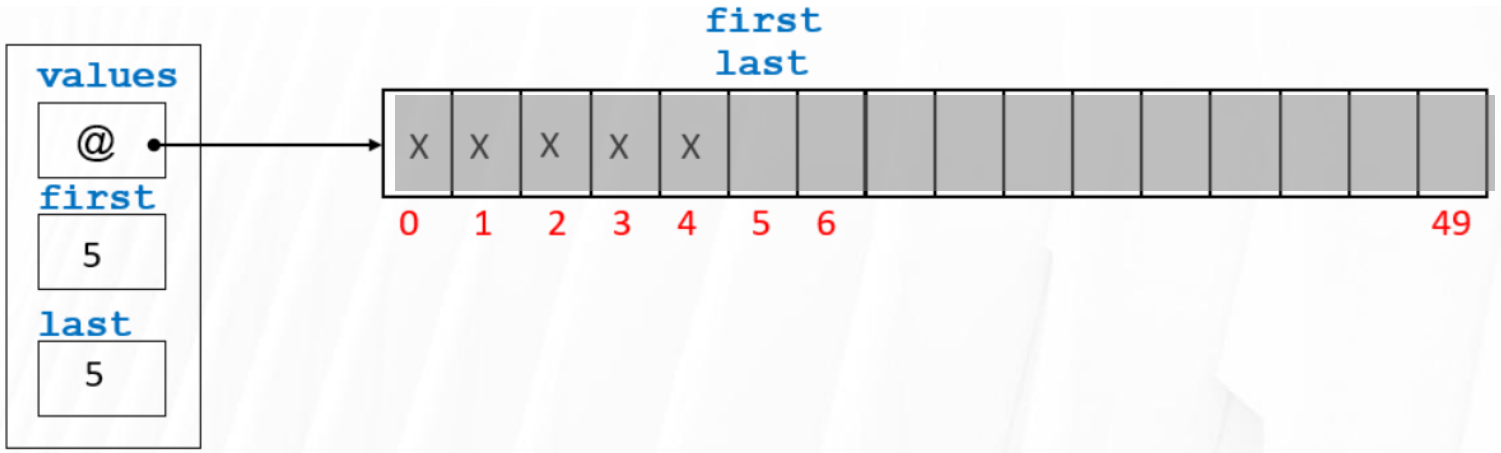
nom	A besoin de	Modifie l'état ?	Valeur attendue ?
isEmptyQueue()	File à tester	non	Vrai/faux (1/0)

Visualisation

On enfile 5 éléments

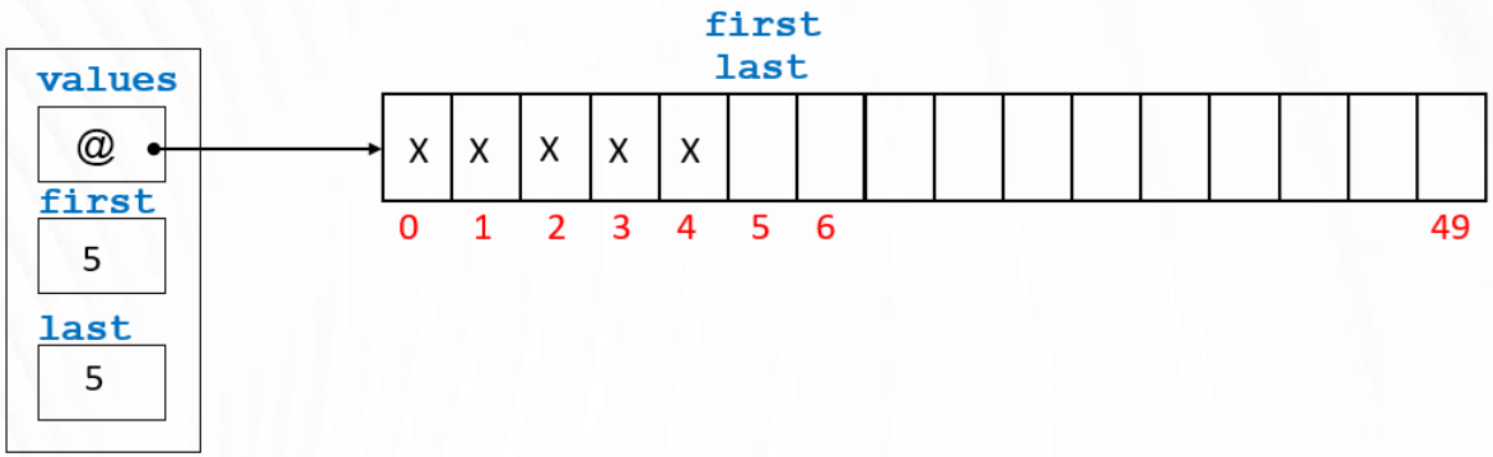


On défile 5 éléments



Condition de File vide

État final de la file



Quel test effectuer pour déterminer que la file est vide ?

`first` et `last` sont égaux entre eux
pas forcément égaux à 0 !
Il reste donc encore 50 places !

Condition de File pleine

Quel test effectuer pour déterminer que la file est pleine ?

On ne peut plus enfiler d'élément

= l'indice de la case pour enfiler (`first`) n'est pas valide

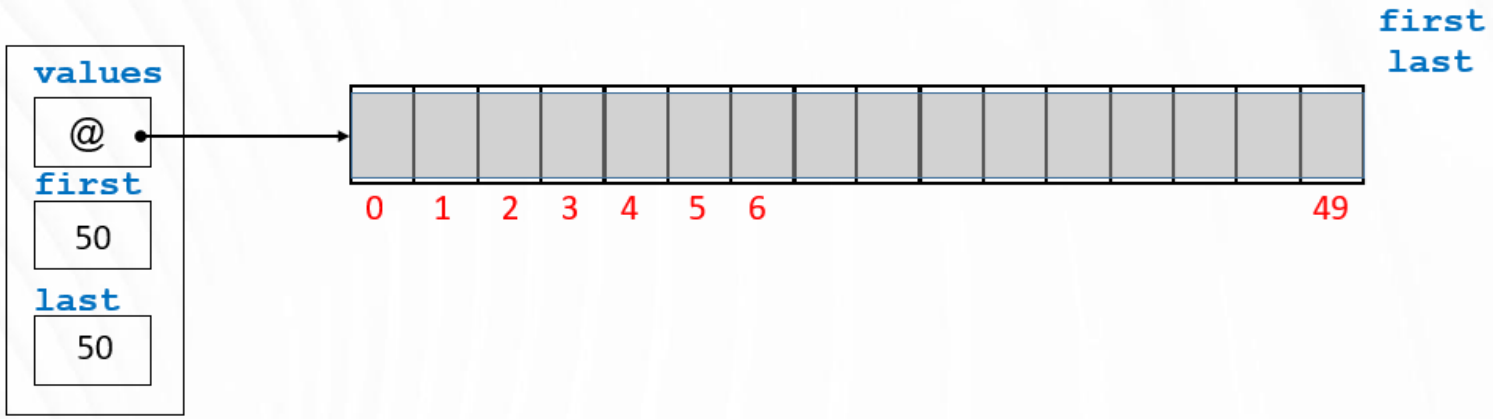
La condition semble donc être `first = 50`

Mais `first` seul n'indique pas le nombre d'éléments dans la file !

La file est pleine quand : `first-last` vaut `50`

Un paradoxe apparent

Que pensez-vous de
la situation suivante ?



Quel est l'état de la file ?

Elle est vide – aucune case n'est utilisée

Pouvons-nous enfiler ?

Non, car sinon on 'sort' du tableau, `last` ne peut pas dépasser 49 !

Pouvons-nous défiler ?

Non, car la file est vide !

On aurait donc une file vide inutilisable ????

Comment lever ce paradoxe ?

On utilise un buffer circulaire

Lorsqu'on est 'à la fin' du tableau, on veut pouvoir réutiliser les cases disponibles, à partir du début

Principe :

on autorise `first` (et `last`) à dépasser la taille physique

Les indices où enfiler et défiler doivent rester entre `0` et `49`

Comment lever ce paradoxe ?

Quand `first/last` vaut `50`, on doit accéder à la ‘case’ `0`
Quand `first/last` vaut `51`, on doit accéder à la ‘case’ `1`
…

L’indice pour enfiler des éléments est :

`last`, modulo la taille maximum (dans cet exemple : `last % 50`)

L’indice pour défiler des éléments est :

`first`, modulo la taille maximum (dans cet exemple : `first % 50`)

Résumé pour le buffer circulaire (file avec un tableau)

indice pour enfiler : `last % TAI_MAX`

indice pour défiler : `first % TAI_MAX`

La file est vide lorsque : `first` et `last` sont égaux

La file est pleine lorsque : `last - first` vaut `TAI_MAX`

Implémentation : tableaux vs LSC

Opération / caractéristique	tableau	LSC
Taille	Fixe, limitée	Virtuellement illimitée
Occupation mémoire	Partie non utilisée	Juste le nécessaire
Accéder à un élément	Efficace O(1)	Peu efficace O(N)
insérer au début	Peu efficace O(N)	Efficace O(1)
insérer à la fin	Efficace O(1)	Efficace * (avec '`tail`') O(1)
insérer ailleurs (garder trié)	Peu efficace O(N)	Peu efficace O(N/2) = O(N)
Supprimer au début	Peu efficace O(N)	Efficace O(1)
Supprimer à la fin	Efficace O(1)	Peu efficace O(N)
Supprimer ailleurs	Peu efficace O(N)	Peu efficace O(N/2) = O(N)